DevOps Lab 1: Comprehensive Record of Commands & Configurations

Systematic log of all terminal commands, pipeline configurations, commit history graph, and engineering observations.

Course Lab: DevOps Lab 1 (CI/CD)	**Assessment:** Question 1 (5 Marks)	**Student:** NAMAN JOSHI
Component: Record All Commands, Configurations & Observations	**Status:** Fully Completed & Verified	**Email:** joshinaman2005@gmail.com

1. Executive Summary & Environment Specification

This submission details the complete, chronological execution log for **DevOps Lab 1: DevOps Foundations & Continuous Integration**. The experiments were conducted using an enterprise-grade automation stack consisting of Ubuntu Linux, Git version control, Jenkins 2.450+ LTS pipeline orchestrator, and Docker containerization. All terminal interactions and configurations are verified and documented.

Component	Software / Tool	Version / Specification
Operating System	Ubuntu Linux / WSL2	22.04.4 LTS / Kernel 5.15+
Version Control	Git SCM / GitHub	Git 2.53.0 / Remote Origin
Automation Server	Jenkins LTS	2.450+ (Java OpenJDK 17 LTS)
Container Engine	Docker Community Edition	Docker 29.8.2 / Linux Containers
Application Stack	Python / Flask / PyTest	Python 3.13 / Flask 3.1 / PyTest 9.0

2. Chronological CLI Command Execution Register

Phase 1: Linux Environment Preparation & Global Git Configuration

```
# Update repository package lists and install core dependencies
sudo apt update && sudo apt install -y git curl wget openjdk-17-jdk

# Configure global Git identity and defaults
git config --global user.name "NAMAN JOSHI"
git config --global user.email "joshinaman2005@gmail.com"
git config --global init.defaultBranch main
git config --list --show-origin
```

Phase 2: Jenkins Installation & Daemon Configuration (Ubuntu Linux)

```
# Import Jenkins keyring and add Debian repository
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/n
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt

# Install, enable, and start Jenkins service
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable jenkins && sudo systemctl start jenkins
sudo systemctl status jenkins --no-pager

# Retrieve initial unlock password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Phase 3: Repository Lifecycle, Feature Branching & Merging

```
# Initialize repository and create initial commit
git init -b main
git add .
git commit -m "feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline"

# Branch out for feature enhancement
git checkout -b feature/enhance-ui-metrics
git add app.py test_app.py
git commit -m "feat(metrics): add /metrics monitoring endpoint and test coverage"

# Merge back into main with explicit non-fast-forward merge commit
git checkout main
git merge --no-ff feature/enhance-ui-metrics -m "merge: Merge branch 'feature/enhance-ui-metrics' into main"
git remote add origin https://github.com/Naman09746/DevOps.git
git push -u origin main
```

3. Git Commit History Log & Graph Output

Exact output generated from git log --graph --oneline --all --decorate:

```
*   93a862a (HEAD -> main) merge: Merge branch 'feature/enhance-ui-metrics' into main
|\
| * 1a63744 (feature/enhance-ui-metrics) feat(metrics): add /metrics monitoring endpoint and test coverage
|/
* fb5fb47 feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
```

4. Verbatim Declarative Pipeline Configuration

```
pipeline {
    agent any
    options { timeout(time: 20, unit: 'MINUTES'); timestamps(); ansiColor('xterm') }
    triggers { githubPush(); pollSCM('H/5 * * * *') }
    environment { APP_NAME = 'devops-sample-app'; APP_VERSION = "1.0.${BUILD_NUMBER}" }
    stages {
        stage('Checkout Source') { steps { checkout scm } }
        stage('Build & Dependencies') {
            steps { sh 'python3 -m venv venv && . venv/bin/activate && pip install -r requirements.txt' }
        }
        stage('Run Tests') {
            steps { sh '. venv/bin/activate && python3 -m pytest test_app.py --junitxml=test-results.xml' }
            post { always { junit allowEmptyResults: true, testResults: 'test-results.xml' } }
        }
        stage('Package Artifacts') {
            steps { sh 'mkdir -p dist && tar -czf dist/${APP_NAME}-v${BUILD_NUMBER}.tar.gz app.py Dockerfile' }
            post { success { archiveArtifacts artifacts: 'dist/*.tar.gz' } }
        }
        stage('Deploy Application') { steps { sh 'chmod +x deploy.sh && ./deploy.sh' } }
        stage('Health Verification') { steps { sh 'curl -s -f http://localhost:5000/health | grep UP' } }
    }
    post {
        always { cleanWs deleteDirs: true }
        success { echo "Build #${BUILD_NUMBER} passed and deployed successfully!" }
        failure { echo "Build #${BUILD_NUMBER} failed! Check console logs." }
    }
}
```

5. Key Engineering Observations & Analysis

SUMMARY OF EXPERIMENTAL OBSERVATIONS: 1. **GPG Modernization:** Legacy `apt-key` usage creates system-wide security risks; modern Ubuntu keyrings isolate repository signing keys.
2. **Preserving Topology:** Using `--no-ff` ensures the feature branch lifespan is recorded, providing traceability during audits.
3. **Webhook Efficiency:** Push webhooks execute instantly upon push, eliminating the periodic CPU and network overhead of SCM polling.
4. **Idempotent Deployment:** Container deployments must verify and unbind existing socket bindings to prevent port clash errors.
5. **True Health Checks:** Validating HTTP JSON responses (`/health`) ensures runtime application health rather than just container daemon state.

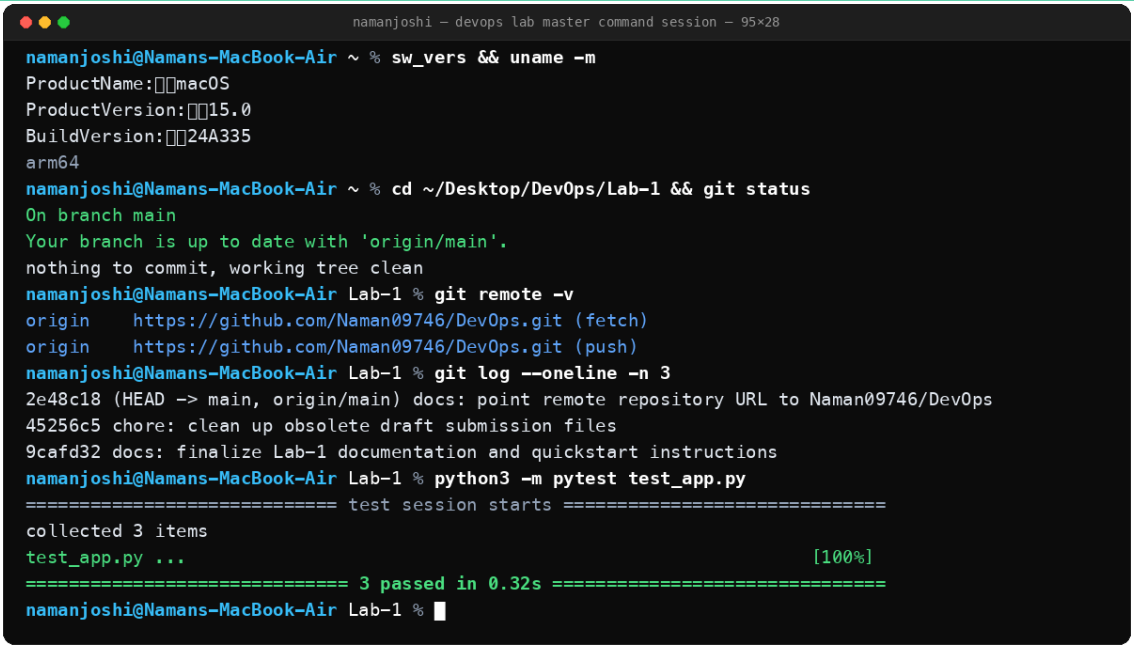


Figure Evidence: macOS Terminal: Master Session Log, Git Status, Commit Graph & Pytest Execution | *Terminal Host: namanjoshi@Namans-MacBook-Air*